```
        for (j=0;j<n;j++) sum += qt[i][j]*b[j];
        x[i] = sum;
    }
}

void QRdcmp::rsolve(VecDoub_I &b, VecDoub_O &x) {
```

Solve the triangular set of `n` linear equations $\mathbf{R} \cdot \mathbf{x} = \mathbf{b}$. `b[0..n-1]` is input as the right-hand side vector, and `x[0..n-1]` is returned as the solution vector.

```
    Int i,j;
    Doub sum;
    if (sing) throw("attempting solve in a singular QR");
    for (i=n-1;i>=0;i--) {
        sum=b[i];
        for (j=i+1;j<n;j++) sum -= r[i][j]*x[j];
        x[i]=sum/r[i][i];
    }
}
```

See [2] for details on how to use QR decomposition for constructing orthogonal bases, and for solving least-squares problems. (We prefer to use SVD, §2.6, for these purposes, because of its greater diagnostic capability in pathological cases.)

2.10.1 Updating a QR decomposition

Some numerical algorithms involve solving a succession of linear systems each of which differs only slightly from its predecessor. Instead of doing $O(N^3)$ operations each time to solve the equations from scratch, one can often update a matrix factorization in $O(N^2)$ operations and use the new factorization to solve the next set of linear equations. The LU decomposition is complicated to update because of pivoting. However, QR turns out to be quite simple for a very common kind of update,

$$\mathbf{A} \to \mathbf{A} + \mathbf{s} \otimes \mathbf{t} \tag{2.10.7}$$

(compare equation 2.7.1). In practice it is more convenient to work with the equivalent form

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{R} \quad \to \quad \mathbf{A}' = \mathbf{Q}' \cdot \mathbf{R}' = \mathbf{Q} \cdot (\mathbf{R} + \mathbf{u} \otimes \mathbf{v}) \tag{2.10.8}$$

One can go back and forth between equations (2.10.7) and (2.10.8) using the fact that $\mathbf{Q}$ is orthogonal, giving

$$\mathbf{t} = \mathbf{v} \quad \text{and either} \quad \mathbf{s} = \mathbf{Q} \cdot \mathbf{u} \quad \text{or} \quad \mathbf{u} = \mathbf{Q}^T \cdot \mathbf{s} \tag{2.10.9}$$

The algorithm [2] has two phases. In the first we apply $N-1$ Jacobi rotations (§11.1) to reduce $\mathbf{R} + \mathbf{u} \otimes \mathbf{v}$ to upper Hessenberg form. Another $N-1$ Jacobi rotations transform this upper Hessenberg matrix to the new upper triangular matrix $\mathbf{R}'$. The matrix $\mathbf{Q}'$ is simply the product of $\mathbf{Q}$ with the $2(N-1)$ Jacobi rotations. In applications we usually want $\mathbf{Q}^T$, so the algorithm is arranged to work with this matrix (which is stored in the `QRdcmp` object) instead of with $\mathbf{Q}$.

```
void QRdcmp::update(VecDoub_I &u, VecDoub_I &v) {
```

qrdcmp.h

Starting from the stored QR decomposition $\mathbf{A} = \mathbf{Q} \cdot \mathbf{R}$, update it to be the QR decomposition of the matrix $\mathbf{Q} \cdot (\mathbf{R} + \mathbf{u} \otimes \mathbf{v})$. Input quantities are `u[0..n-1]`, and `v[0..n-1]`.

```
    Int i,k;
    VecDoub w(u);
    for (k=n-1;k>=0;k--)
        if (w[k] != 0.0) break;
    if (k < 0) k=0;
    for (i=k-1;i>=0;i--) {
        rotate(i,w[i],-w[i+1]);
        if (w[i] == 0.0)
            w[i]=abs(w[i+1]);
```

Find largest `k` such that `u[k]` $\neq 0$.

Transform $\mathbf{R} + \mathbf{u} \otimes \mathbf{v}$ to upper Hessenberg.

```
        else if (abs(w[i]) > abs(w[i+1]))
            w[i]=abs(w[i])*sqrt(1.0+SQR(w[i+1]/w[i]));
        else w[i]=abs(w[i+1])*sqrt(1.0+SQR(w[i]/w[i+1]));
    }
    for (i=0;i<n;i++) r[0][i] += w[0]*v[i];
    for (i=0;i<k;i++)                    Transform upper Hessenberg matrix to upper tri-
        rotate(i,r[i][i],-r[i+1][i]);      angular.
    for (i=0;i<n;i++)
        if (r[i][i] == 0.0) sing=true;
}

void QRdcmp::rotate(const Int i, const Doub a, const Doub b)
```

Utility used by `update`. Given matrices `r[0..n-1][0..n-1]` and `qt[0..n-1][0..n-1]`, carry out a Jacobi rotation on rows `i` and `i + 1` of each matrix. `a` and `b` are the parameters of the rotation: $\cos\theta = a/\sqrt{a^2+b^2}$, $\sin\theta = b/\sqrt{a^2+b^2}$.

```
{
    Int j;
    Doub c,fact,s,w,y;
    if (a == 0.0) {                  Avoid unnecessary overflow or underflow.
        c=0.0;
        s=(b >= 0.0 ? 1.0 : -1.0);
    } else if (abs(a) > abs(b)) {
        fact=b/a;
        c=SIGN(1.0/sqrt(1.0+(fact*fact)),a);
        s=fact*c;
    } else {
        fact=a/b;
        s=SIGN(1.0/sqrt(1.0+(fact*fact)),b);
        c=fact*s;
    }
    for (j=i;j<n;j++) {              Premultiply r by Jacobi rotation.
        y=r[i][j];
        w=r[i+1][j];
        r[i][j]=c*y-s*w;
        r[i+1][j]=s*y+c*w;
    }
    for (j=0;j<n;j++) {              Premultiply qt by Jacobi rotation.
        y=qt[i][j];
        w=qt[i+1][j];
        qt[i][j]=c*y-s*w;
        qt[i+1][j]=s*y+c*w;
    }
}
```

We will make use of QR decomposition, and its updating, in §9.7.

CITED REFERENCES AND FURTHER READING:

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I/8.[1]

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §5.2, §5.3, §12.5.[2]

2.11 Is Matrix Inversion an N^3 Process?

We close this chapter with a little entertainment, a bit of algorithmic prestidigitation that probes more deeply into the subject of matrix inversion. We start with a seemingly simple question:

How many individual multiplications does it take to perform the matrix multiplication of two 2×2 matrices,

$$\begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix} \cdot \begin{pmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{pmatrix} = \begin{pmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{pmatrix} \tag{2.11.1}$$

Eight, right? Here they are written explicitly:

$$\begin{aligned} c_{00} &= a_{00} \times b_{00} + a_{01} \times b_{10} \\ c_{01} &= a_{00} \times b_{01} + a_{01} \times b_{11} \\ c_{10} &= a_{10} \times b_{00} + a_{11} \times b_{10} \\ c_{11} &= a_{10} \times b_{01} + a_{11} \times b_{11} \end{aligned} \tag{2.11.2}$$

Do you think that one can write formulas for the c's that involve only *seven* multiplications? (Try it yourself, before reading on.)

Such a set of formulas was, in fact, discovered by Strassen [1]. The formulas are

$$\begin{aligned} Q_0 &\equiv (a_{00} + a_{11}) \times (b_{00} + b_{11}) \\ Q_1 &\equiv (a_{10} + a_{11}) \times b_{00} \\ Q_2 &\equiv a_{00} \times (b_{01} - b_{11}) \\ Q_3 &\equiv a_{11} \times (-b_{00} + b_{10}) \\ Q_4 &\equiv (a_{00} + a_{01}) \times b_{11} \\ Q_5 &\equiv (-a_{00} + a_{10}) \times (b_{00} + b_{01}) \\ Q_6 &\equiv (a_{01} - a_{11}) \times (b_{10} + b_{11}) \end{aligned} \tag{2.11.3}$$

in terms of which

$$\begin{aligned} c_{00} &= Q_0 + Q_3 - Q_4 + Q_6 \\ c_{10} &= Q_1 + Q_3 \\ c_{01} &= Q_2 + Q_4 \\ c_{11} &= Q_0 + Q_2 - Q_1 + Q_5 \end{aligned} \tag{2.11.4}$$

What's the use of this? There is one fewer multiplication than in equation (2.11.2), but *many more* additions and subtractions. It is not clear that anything has been gained. But notice that in (2.11.3) the a's and b's are never commuted. Therefore (2.11.3) and (2.11.4) are valid when the a's and b's are themselves matrices. The problem of multiplying two very large matrices (of order $N = 2^m$ for some integer m) can now be broken down recursively by partitioning the matrices into quarters, sixteenths, etc. And note the key point: The savings is not just a factor "7/8"; it is that factor at *each* hierarchical level of the recursion. In total it reduces the process of matrix multiplication to order $N^{\log_2 7}$ instead of N^3.

What about all the extra additions in (2.11.3) – (2.11.4)? Don't they outweigh the advantage of the fewer multiplications? For large N, it turns out that there are six times as many additions as multiplications implied by (2.11.3) – (2.11.4). But, if N is very large, this constant factor is no match for the change in the *exponent* from N^3 to $N^{\log_2 7}$.